DeciMark

Workflow Justification & Reflections on Collaborative Development

ITEC50 Final Project — Individual Submission

A documentation of technical decisions, professional standards, and the realities of collaborative development

Lyra Phasma

ITEC50

Table of Contents

Part I

Research

1 Background Research & Competitive Analysis

1.1 Website Type & Purpose

DeciMark is a Bookmark Link Manager—a web application that enables users to save, organize, retrieve, and annotate URLs. According to Abrams et al. (1998), it belongs to the broader category of Personal Information Management (PIM) systems, specifically the subcategory of social and personal bookmarking tools.

The application is server-backed with a full database layer and requires a user account. No client-side installation is required: users access DeciMark entirely through a browser, with all infrastructure complexity contained on the server side. In later development stages, DeciMark is planned to be distributable as a portable installable—`.exe`, `.msi`, or `.AppImage`—for users who prefer self-hosting without a full server environment.

1.1.1 Why Bookmark Management Is Essential in Professional Contexts

In the modern knowledge economy, information workers routinely navigate dozens of shared digital resources daily: vendor portals, internal wikis, project dashboards, compliance references, design assets, and client-facing URLs. Without a structured system, these links scatter across browser bookmarks, messaging threads, spreadsheets, and email histories—invisible to collaborators and impossible to audit.

A McKinsey Global Institute study found that knowledge workers spend an estimated **20% of their working week** searching for information or tracking down colleagues who can help locate it (Chui et al., July 2012). A centralized, structured bookmark manager directly reduces this friction. In a professional team context, it functions as an operational backbone for the following use cases:

- **Research teams** cataloguing literature, citations, and reference links by project phase
- **Developers** tracking documentation pages, API references, and deployment dashboards
- **Business analysts** managing client portals, third-party SaaS tools, and reporting endpoints
- **Students** organizing academic resources, course links, reading lists, and submission portals

> Users spend most of their time on other sites. This means that users prefer your site to work the same way as all the other sites they already know.
>
> Pernice (2014)

This principle extends to information architecture: users expect to find things where they expect them. A bookmark manager without a coherent organizational scheme violates this expectation at scale.

1.2 Existing Professional Examples

The following two platforms were selected as primary reference points because they represent opposite ends of the bookmark manager design spectrum: one prizing radical minimalism, the other prizing self-sovereignty.

1.2.1 Pinboard.in

Pinboard (https://pinboard.in) is one of the oldest surviving social bookmark managers on the web, founded in 2009 by Maciej Cegłowski as a deliberate antidote to the bloat of its contemporaries (Cegłowski, July 2009). It gained significant adoption when the formerly dominant Delicious bookmarking service was acquired and effectively shuttered by Yahoo! in 2010, driving thousands of users to seek alternatives (Singel, December 2010).

Core Features

- Tag-based flat organization (no folders or hierarchy)
- Optional archive mode—saves a cached copy of bookmarked pages (paid tier)
- Public and private bookmark visibility toggles
- Minimalist, text-only interface explicitly designed for speed over aesthetics
- REST API for third-party application integration
- Annual subscription model ($11/year for new accounts as of 2015) (Cegłowski, July 2015)

Identified Gaps & Weaknesses

Pinboard's design philosophy of deliberate minimalism, while coherent, produces several structural gaps that our project addresses:

- **No visual interface.** Pinboard renders as a plain text list with no thumbnails, link

previews, or card-based layouts. This imposes a high cognitive load on users managing large collections, as links are distinguished only by title text and tags (Jones, 2007).

- **Flat tag system.** Pinboard offers no hierarchical organization—no folders, no categories, no enforced naming schema. Tags are entirely free-form, meaning any two users will inevitably develop incompatible taxonomies for the same content domain.
- **No structured classification.** There is no concept of a numeric identifier, reference code, or stable address for any given bookmark beyond its URL.
- **Poor mobile responsiveness.** The interface pre-dates mobile-first design paradigms and has not been substantially updated.
- **No collaborative features.** Beyond making individual pins public, Pinboard offers no team workspaces, shared collections, or multi-user permission systems.

Most critically: as of late 2024, Pinboard entered a period of significant operational decline. The service is maintained by a single developer who became largely unresponsive to support inquiries, with users reporting increasing downtime, throttled API access, and no public roadmap (Tsai, September 2024). Long-time subscribers publicly migrated to alternatives.

> “Pinboard is a one person show and that person is no longer responding to support emails. Maciej is no longer active on social media that I know of. His Pinboard.in support forum has been quiescent for years.
>
> Michael Tsai Tsai (September 2024) ”

This is a meaningful lesson in the fragility of single-maintainer SaaS dependencies—and a direct motivation for DeciMark’s self-hostable architecture.

1.2.2 linkding

linkding (https://linkding.link) is a self-hosted, open-source bookmark manager built with the Django web framework and designed to be deployed via Docker (Sissbruecker, 2020). It is minimally scoped by design: fast, auditable, and entirely under the operator’s control.

Core Features

- Full self-hosting: the operator owns all data and infrastructure
- Tag-based bookmark organization

- Browser extension for one-click saving
- REST API for automation and integrations
- Netscape HTML bookmark import and export
- Multi-user support via Django's admin panel
- Snapshot archiving via `singlefile` integration

Identified Gaps & Weaknesses

- **Requires Docker and server infrastructure.** linkding is inaccessible to any user without Linux/server administration experience. For a student team or a small business without a DevOps resource, the setup cost is prohibitive (Sissbruecker, 2020).
- **No visual card or grid view.** The interface is list-only. Link previews and thumbnail generation are absent.
- **Flat tagging, no structural schema.** Like Pinboard, linkding uses free-form tags without any enforced hierarchy or classification model.
- **Zero onboarding for general users.** The project is explicitly developer-targeted; there is no guided setup, no hosted version, and no consumer-facing documentation.
- **No link metadata enrichment.** Beyond a title and description scraped at save-time, no visual preview or favicon resolution is performed.

1.2.3 Raindrop.io (Additional Reference)

Raindrop.io (https://raindrop.io) was examined as a third reference point representing the premium, cloud-hosted segment of the market. It offers a substantially more visual and feature-rich experience than either Pinboard or linkding (Raindrop.io, 2024).

Core Features

- Nested collections (folders with sub-folders)
- Multiple view modes: Grid, Masonry, Headlines, List
- Tag support alongside folder hierarchy
- Browser extensions, iOS/Android apps, Windows/macOS desktop apps

- Broken-link and duplicate detection
- Full-text search within saved pages (Pro tier)
- Annotation and highlighting tools

Identified Gaps & Weaknesses

- **No structured numeric classification.** Folder naming is entirely user-defined with no enforced schema. Two team members will inevitably use different naming conventions (Jones, 2007).
- **Key features are paywalled.** Nested collections, full-text search, and permanent caching require a Pro subscription at $28/year, which is a barrier for student use (Raindrop.io, 2024).
- **Cloud-only, no self-hosting option.** Raindrop.io has no self-hostable mode; all data is held on their servers with no operator control.
- **Overengineered for constrained use cases.** For a student team or small business needing only structured, consistent categorization, Raindrop.io introduces organizational surface area without enforcing organizational discipline.

1.3 Competitor Comparison Matrix

Table 1.1 summarizes the feature gaps across all three reference platforms relative to our project's design goals.

Table 1.1: Competitor Feature Comparison Matrix

Feature	**Pinboard**	**linkding**	**Raindrop.io**	**DeciMark (Ours)**
Visual card UI	None	None	Grid/Masonry	Card + List toggle
Org. model	Flat tags	Flat tags	Folders + tags	JD IDs + tags + folders
JD numeric system	None	None	None	**Built-in**
User accounts	Yes	Yes (admin)	Yes	Yes
Client-side install	None required	None required	None required	None required
Self-hostable	No	Yes (Docker)	No	Yes
Service reliability	Declining (2024)	Stable	Stable	Self-hosted
Mobile responsive	Poor	Basic	Good	Mobile-first
Portable distribution	No	No	No	Planned (.exe/.AppImage)
Collaborative use	Public pins only	Multi-user (admin)	Pro tier	Scoped by JD area

1.4 My Project: DeciMark

1.4.1 Design Philosophy & Differentiation

DeciMark does not attempt to replicate the full feature surface of any single existing tool. Instead, it occupies a deliberate gap: a visually structured, schema-enforced, self-hostable bookmark manager that any user can access through a browser without installing anything on their machine—while giving operators full control over their own data and infrastructure.

The three founding design principles are:

1. **Structural discipline over organizational freedom.** Free-form tags and arbitrary folder names produce entropy over time. DeciMark enforces the Johnny.Decimal classification scheme as a first-class field on every bookmark, ensuring consistent addressability across users and sessions.

2. **Zero client-side friction.** The application is accessed through a browser. End users install nothing. All infrastructure complexity lives on the server, where it belongs.

3. **Visual clarity without visual bloat.** A card-based layout with list-view toggle provides the visual orientation that Pinboard and linkding lack, without the complexity of Raindrop.io's full platform.

1.4.2 Inspiration: The Johnny.Decimal System

The primary structural innovation in DeciMark is the integration of the **Johnny.Decimal** (JD) organizational system, developed by Johnny Noble (https://johnnydecimal.com) (Noble, 2021).

The JD system assigns every item in a collection a unique two-part numeric identifier in the format `AC.ID`, where:

- The **Area** (`10--19`, `20--29`, …) groups broad domains of activity into sets of ten categories
- The **Category** (`15`, `32`, …) groups related items within an area, with a maximum of ten categories per area
- The **ID** (`15.23`, `32.04`, …) uniquely identifies the specific item within its category

> "You assign a unique ID to everything in your life. These IDs help you stay organised. They impose constraints that make it harder to get lost."
>
> Noble (2021)

Example application to bookmarks:

A link to a vendor's API documentation might be assigned `32.04`, placing it in Area `30-39` ("Engineering & Tools"), Category `32` ("External APIs"), as the fourth item in that category.

This has several concrete advantages over free-form tagging:

- **IDs are stable.** Unlike folder names that shift with renaming, a JD number does not move. `32.04` always refers to the same thing.
- **IDs are speakable.** "Check thirty-two oh-four" is faster to communicate than "it's in the Engineering folder, under External APIs, find the vendor docs."
- **IDs impose productive constraints.** A maximum of 10 categories per area and 99 items

per category forces regular pruning and prevents organizational sprawl (Noble, 2021).

- **IDs are sortable and filterable.** The numeric schema enables deterministic sorting and range-based filtering (e.g., display all items in Area `30-39`).

No existing bookmark manager on the market integrates this system as a native field. This is the primary differentiating feature of DeciMark.

1.4.3 Features Adopted from Existing Tools

Where existing tools already solve a problem well, we adopt rather than reinvent:

- **Tag support**—adopted from Pinboard and linkding. Free-form tags complement the structured JD IDs and support cross-cutting retrieval that the numeric scheme alone cannot express.
- **Card-based visual layout**—inspired by Raindrop.io's grid view, which we identified as the most effective at-a-glance organization model. This is implemented as the primary view mode.
- **Multi-attribute search and filter**—combining linkding's keyword search speed with Raindrop.io's filtering by multiple attributes (tag, JD ID range, keyword).
- **Minimalist interface**—Pinboard's commitment to speed and the absence of unnecessary UI chrome informed our visual design restraint.
- **User accounts and persistent data**—adopted from linkding and Raindrop.io. Bookmark collections are tied to authenticated user accounts and persisted in a database, enabling access from any device without client-side storage.

1.4.4 Features Deliberately Excluded

The following features present in existing tools were intentionally omitted from the current MVP scope:

- **Browser extension**—extension APIs are browser-specific and platform-dependent, placing them outside the current implementation scope. Planned for a future release.
- **Full-text page archiving**—requires a server-side crawler with significant storage overhead. Out of scope for the current iteration.
- **Portable distribution**—`.exe`, `.msi`, and `.AppImage` packaging is planned for later

development stages to support self-hosting without a full server environment.

Part II

Programmatically Generated Reports

2 Source Lines of Code

2.1 Code Statistics

Generated by *scc* (Sloc Cloc and Code). 2026-05-25 18:02

Table 2.1: Lines of Code by Language

Language	Files	Lines	Code	Comments	Blank	Bytes	Complexity
Python	38	4,110	3,431	195	484	138,828	263
Markdown	24	826	581	0	245	30,079	0
TeX	20	1,589	1,106	103	380	91,330	68
CSS	17	27,460	20,623	894	5,943	429,915	0
HTML	12	751	712	21	18	31,921	0
JSON	8	426	420	0	6	11,618	0
JavaScript	8	705	615	5	85	21,907	86
SVG	2	19	19	0	0	1,664	0
YAML	2	522	517	4	1	26,563	0
INI	1	44	29	5	10	814	0
Mako	1	29	21	0	8	733	0
Shell	1	49	36	6	7	1,518	6
TOML	1	210	187	3	20	3,653	0
Total	**135**	**36,740**	**28,297**	**1,236**	**7,207**	**790,543**	**423**

The project spans **135 files** across **13 languages**, totalling **36,740 lines** (28,297 lines of code) and **790,543 bytes** on disk.

2.2 COCOMO Estimates

Table 2.2: Organic COCOMO Estimates

Metric	Estimate
Estimated Cost to Develop	$903,488.73
Estimated Schedule Effort	13.23 months
Estimated People Required	6.07

3 CSS File Strcture

3.1 base.back.css

Source: `src/static/stylesheets/base.back.css`

Generated: 2026-05-25 19:12

3.1.1 Lint Results

No lint issues found.

3.1.2 Section Tree

```
.
├── pushes footer to the bottom if content is short
└── big whitespace at the end
```

3.2 base.css

Source: `src/static/stylesheets/base.css`

Generated: 2026-05-25 19:12

3.2.1 Lint Results

No lint issues found.

3.2.2 Section Tree

```
.
├── IMPORTS  (line 1-4)
├── VARIABLES  (line 6-194)
```

```
│   ├── GENERAL  (line 9-16)
│   ├── COLORS  (line 18-93)
│   │   ├── BACKGROUND  (line 20-46)
│   │   │   ├── #1  (line 22-34)
│   │   │   └── #2  (line 36-44)
│   │   ├── FOREGOUND  (line 48-80)
│   │   │   ├── #1  (line 50-59)
│   │   │   ├── #2  (line 61-70)
│   │   │   └── #3  (line 72-78)
│   │   ├── STATUS  (line 82-86)
│   │   └── EFFECTS  (line 88-91)
│   └── SPACES  (line 95-127)
│       ├── STANDALONE  (line 97-108)
│       ├── ONE-UP PAIRS  (line 110-120)
│       └── CUSTOM PAIRS  (line 122-125)
├── base: hsl(230, 25%, 10%)
├── base: hsl(232, 22%, 40%);
├── base: hsl(260, 50%, 70%);
├── base: hsl(326, 70%, 70%);
├── base: hsl(321, 100%, 81%)
├── COLOR  (line 196-206)
├── override `./downloaded/browserux.css`
├── HEADERS  (line 208-261)
├── ICONS  (line 263-281)
├── SVG ICONS  (line 283-302)
├── BUTTONS  (line 304-348)
├── STRUCTURE  (line 350-388)
├── VISIBILITY  (line 390-398)
├── SCROLLBAR  (line 400-423)
│   ├── MAIN
│   ├── HANDLE
│   ├── HANDLE ON HOVER
│   └── TRACK
├── OTHER  (line 425-440)
│   └── STRIKETHROUGH
└── HAMBURGER MENU  (line 442-484)
```

3.3 bookmarks.css

Source: src/static/stylesheets/bookmarks.css

Generated: 2026-05-25 19:12

3.3.1 Lint Results

No lint issues found.

3.3.2 Section Tree

```
(no sections)
```

3.4 code.css

Source: `src/static/stylesheets/code.css`

Generated: 2026-05-25 19:12

3.4.1 Lint Results

No lint issues found.

3.4.2 Section Tree

```
└── RESPONSIVE CSS  (line 83-125)
```

3.5 fonts/comic-mono.css

Source: `src/static/stylesheets/fonts/comic-mono.css`

Generated: 2026-05-25 19:12

3.5.1 Lint Results

No lint issues found.

3.5.2 Section Tree

```
(no sections)
```

3.6 fonts/inconsolata.css

Source: `src/static/stylesheets/fonts/inconsolata.css`

Generated: 2026-05-25 19:12

3.6.1 Lint Results

No lint issues found.

3.6.2 Section Tree

```
(no sections)
```

3.7 fonts/roboto.css

Source: `src/static/stylesheets/fonts/roboto.css`

Generated: 2026-05-25 19:13

3.7.1 Lint Results

No lint issues found.

3.7.2 Section Tree

```
(no sections)
```

3.8 forms.css

Source: `src/static/stylesheets/forms.css`

Generated: 2026-05-25 19:13

3.8.1 Lint Results

No lint issues found.

3.8.2 Section Tree

```
.
├── FORM
├── FORM BOX
└── INPUTS  (line 14-94)
```

3.9 modal.css

Source: `src/static/stylesheets/modal.css`

Generated: 2026-05-25 19:13

3.9.1 Lint Results

No lint issues found.

3.9.2 Section Tree

```
(no sections)
```

3.10 nuke.css

Source: `src/static/stylesheets/nuke.css`

Generated: 2026-05-25 19:13

3.10.1 Lint Results

No lint issues found.

3.10.2 Section Tree

```
└── NUCLEAR RESET  (line 1-177)
```

3.11 responsive-base.css

Source: `src/static/stylesheets/responsive-base.css`

Generated: 2026-05-25 19:13

3.11.1 Lint Results

No lint issues found.

3.11.2 Section Tree

```
(no sections)
```

3.12 responsive-bg.css

Source: `src/static/stylesheets/responsive-bg.css`

Generated: 2026-05-25 19:13

3.12.1 Lint Results

No lint issues found.

3.12.2 Section Tree

```
(no sections)
```

3.13 toast.css

Source: `src/static/stylesheets/toast.css`

Generated: 2026-05-25 19:13

3.13.1 Lint Results

No lint issues found.

3.13.2 Section Tree

```
(no sections)
```

Part III

Reflection

4 Preface to the Reflection

I want to be upfront about something before you read any of this: I was not always this way about coding.

There was a time when I did it purely because I loved it. Open source contributions, small projects, learning something new every day — not because anyone needed me to, not because my medication costs 700 pesos every two days and someone has to cover it, but just because it was fun. It was my first love. And then, because of circumstances largely outside my control, I was pushed to monetize it before I was ready. It became survival. And somewhere in that transition, I lost the plot a little.

I say this because it explains, at least partially, why I reacted the way I did to a group of first-year students I had known for less than 48 hours.

They had the sparkle. The ambition, the optimism, the whole universe of ideas and dreams that comes with being at the beginning of something. And I, having lost that somewhere along the way, did not handle it gracefully. I saw their optimism and called it naivety. I saw their excitement and called it poor scope management. And I was not entirely wrong — the technical concerns in this document are real — but I was not entirely right either, because being technically correct is not the same as being a good collaborator.

Here is what I genuinely regret: I robbed myself of the opportunity to actually collaborate with them. To remember what it felt like to be excited about something for no reason other than the excitement itself. Instead, I got rigid, I got frustrated, and I got clinical about it — which, in fairness, produced this document, so it was not entirely unproductive. But still.

They hurt my credibility by dismissing concerns I had earned the right to raise. I hurt their confidence by treating their beginning like a deficiency. The damage is done, on both sides, and I own my share of it.

What I can say is this: the arguments that follow are honest, and the engineering principles are sound, and the self-reflection is as genuine as I could make it. But they were also written by someone who is tired, and jaded, and still quietly grieving a version of herself that coded for fun — and who took that grief out, at least partly, on a group of first-year students who just wanted to build something together.

They were not wrong to want that. I just forgot, briefly, that wanting to build something together was once enough of a reason.

5 Reflection

5.1 On Version Control in Collaborative Development

Version control systems such as Git are not merely organizational tools — they are the structural backbone of any collaborative software project. Without them, teams default to informal file-sharing workflows that introduce critical failure points: no single source of truth, no conflict resolution mechanism, and no traceability of changes. This is not a matter of technical preference. It is a matter of project viability.

The industry has long since settled this debate. Git is listed as a baseline requirement in the overwhelming majority of software engineering job listings — not as a bonus skill, but as a prerequisite. A developer without Git literacy is, professionally speaking, an incomplete one.

5.2 On the Challenges of Informal File-Sharing Workflows

File-sharing via cloud drives or messaging platforms creates what practitioners call integration hell — a state where independently developed components cannot be cleanly merged. CSS conflicts cascade unpredictably. JavaScript logic written in isolation breaks when combined with another developer's structural assumptions. Without diff tracking, identifying the source of these conflicts becomes exponentially harder as the codebase grows.

Consider a concrete scenario: two developers edit the same stylesheet simultaneously via Google Drive. Neither knows the other is working. One changes the `.container` class to use CSS Grid. The other adds Flexbox properties to the same selector. When merged manually — via copy-paste — one set of changes is lost entirely, with no record of what was overwritten, by whom, or why. This is not a hypothetical. This is the default outcome of unstructured collaboration.

Messenger-based code sharing compounds this further. Code loses formatting. Context is lost. There is no version history. Rolling back a breaking change requires manually reconstructing previous states from chat logs — if they have not already been deleted.

5.3 On Scope Management and Realistic Assessment

Scope discipline is not an advanced skill. It is a foundational one. The ability to match project ambitions to available time, skillset, and resources is not taught exclusively in senior-level courses or professional environments — it is the first lesson of any serious creative or technical endeavor. To ignore it is not boldness. It is not ambition. It is the absence of self-awareness, disguised as ambition so that failure can be mistaken for effort rather than recognized as misjudgment.

The projects proposed by the original group included features characteristic of enterprise-level systems — social media platforms with real-time messaging, e-commerce infrastructure, and large-scale inventory management. These are not beginner projects. They are the outputs of seasoned engineering teams operating over months, with dedicated backend infrastructure, database architecture, and — critically — version control. To propose them within a four-week timeframe, with a first-year team, without backend experience, and without structured collaboration tooling, is not ambitious. It is a failure of assessment.

This concern was not raised in isolation. An industry professional with full-stack expertise — consulted specifically about the group's proposed scope — was unambiguous in his assessment:

> "Even in a group, in such a grand scale, won't be able to finish the project in 4 weeks. Especially since you are planning to spend the first week of it on brainstorming ideas."

This feedback was relayed to the team directly. It was not received.

What followed was perhaps more revealing than the scope itself. When confronted with the gap between their ambitions and their current skillset, the response was not recalibration. It was resignation reframed as philosophy: that being first years was sufficient justification for not engaging with the technical realities of the project. That uncertainty about continuing the course absolved them of the responsibility to learn within it.

This is where scope failure becomes something more fundamental than poor planning. Software development — and computer science broadly — is a discipline that demands learning beyond the four corners of the classroom. The tools, frameworks, and professional standards

that define the field are not delivered exclusively through formal instruction. They are pursued. A team unwilling to pursue them is not limited by their first-year status. They are limited by their own ceiling — one they have chosen to set remarkably low.

Grit is not a personality trait reserved for the exceptional. It is the baseline requirement of any discipline worth practicing. And it cannot be substituted by group cohesion, shared optimism, or the comfortable certainty that failure, if it comes, will at least be experienced together.

5.4 On the Engineering Principles That Were Never Discussed

Somewhere in the noise of escalating conflict, the actual engineering principles underlying this disagreement were entirely lost. This section retrieves them.

5.4.1 Minimum Viable Product

The most fundamental mistake in scope-setting was the conflation of ambition with demonstrability. In any software project, the first question is not “What would be incredible to build?” but “What is the smallest thing that proves the concept?” This is the Minimum Viable Product — a term so universal in the discipline that it requires no defense.

A social media platform with real-time messaging, search, and a recommendation algorithm is not an MVP for a team of first-year students working across four weeks without backend infrastructure. It is the output of a funded engineering team operating across months. The MVP in this context was a static demonstration — something that showed core functionality without requiring any of the infrastructure the group was unwilling to learn. The contributor advocated for exactly this. The group did not adopt it until after the contributor had already left, and after direct, unmediated contact with the complexity of the original ideas confirmed what had been said from the outset.

5.4.2 Failing Fast Requires a Feedback Loop

The philosophy of fail fast, learn faster was implicitly invoked throughout the group’s defense of their approach — the idea that shared experience of difficulty would ultimately strengthen both the team and its output. This is a real principle. It is not, however, unconditional.

Failing fast works only when there exists a mechanism to capture why a failure occurred. Without version control, diff tracking, change attribution, and rollback capability, a team fails repeatedly but learns nothing reproducible. It suffers the same mistake in slightly different forms, bonds over the suffering, and mistakes the bond for the lesson.

Messenger-based code sharing is the precise environment in which this dynamic unfolds. If two contributors edit the same stylesheet and one's changes silently overwrite the other's, the failure has no name, no author, and no recoverable history. The code breaks. Time is lost. The cause is guessed at. That is not failing fast. That is failing blind.

5.4.3 Conway's Law

A structurally important principle runs through all of this and deserves naming: Conway's Law states that organizations design systems that mirror their own communication structures. A team that communicates through disorganized fragments in a group chat — where messages are lost, tone is hostile, and decisions are deferred through inertia — will inevitably produce software that is itself disorganized, fragile, and resistant to integration.

The contributor's insistence on Git and structured scope management was, in this light, not merely a technical preference. It was an attempt to establish a communication structure that would produce coherent technical output. The group's resistance to that structure ensured that the communication environment remained precisely the kind that Conway's Law would predict as productive only of conflict and confusion.

The repository that now exists — `Peak-ass-repository-ng-mga-maiitim-na-komsay` — is a small monument to the principle. Its name reflects the communication culture from which it emerged. Its existence reflects the belated recognition that communication structure and software structure are, ultimately, the same thing.

5.5 On the Use of Artificial Intelligence as a Workflow Tool

A note on the use of AI in this project, because it deserves honest treatment rather than either defensive dismissal or uncritical enthusiasm.

The author used AI. Deliberately, selectively, and without apology.

This is worth explaining, because the difference between AI as a crutch and AI as a force multiplier is not a difference in the tool — it is a difference in the person using it. And that difference comes down to one thing: knowing what you do not know, and knowing what is and is not worth your time.

Consider a concrete example from the author's own workflow. At one point in this project, FontAwesome needed to be integrated into the codebase. The author knew exactly what needed to happen:

- Fetch the latest non-prerelease FontAwesome Webfonts and CSS release from the GitHub API
- Extract the correct zip archive
- Copy `css/all.css` to the project's stylesheet directory
- Copy the contents of `webfonts/` to the static assets folder
- Rewrite all `url("../webfonts/")` references in the CSS to `url("/static/assets/fonts/")` to match FastAPI's static file serving path

The author knew all of this. She knew the directory structure to expect, the path rewriting required, and exactly why it was necessary. She also knew that writing a shell script to automate it was not a problem worth her full attention — it was a utility task, a cherry on top, a nice-to-have automation that sat firmly below the actual work on the priority list.

So she described the task precisely and delegated it.

This is not a reduction in intelligence. This is the correct application of it. The skill on display is not the ability to write a download script. It is the ability to specify, with enough precision that the task can be delegated without ambiguity, exactly what needs to happen and why. That specification — knowing the API endpoint, the file structure, the serving path, the CSS rewrite — is the intellectual work. The script is just the output.

This is the bulk of how AI appears in this codebase: utilities, small snippets, automation for tasks whose shape is known and whose implementation is not the point. It does not appear in architectural decisions. It does not appear in the core logic. It does not appear anywhere that understanding matters more than output — because in those places, delegating the thinking is precisely what makes the thinking worse.

The distinction is one that takes time and experience to develop. It requires knowing, first, what you are trying to build and why. It requires understanding the system well enough to know which parts deserve your full attention and which parts are just plumbing. It requires, in short, knowing what you do not know — and knowing what is beneath your pay grade to implement by hand.

This is not a skill that arrives automatically. It is earned, usually by doing things the hard way first, until the shape of the problem is familiar enough that the implementation becomes obvious.

A team that has not yet built that familiarity will not use AI to accelerate their workflow. They will use it to substitute for the understanding they have not yet developed — which produces output without comprehension, and comprehension, ultimately, is the only thing that compounds.

The author is not saying this to be unkind. She is saying it because she was there once. The hard way is how the shape of things becomes familiar. There are no shortcuts to that particular kind of knowing.

AI is an excellent tool. It is most excellent in the hands of someone who already knows what they are doing.

5.6 On the Cost of Refusing Structured Tooling

The resistance to adopting version control in a collaborative project is not simply a workflow preference. It carries compounding costs:

- Conflicts become invisible until they break something.
- There is no mechanism to attribute changes, making debugging exponentially harder.
- Rollback is impossible without manual reconstruction.
- Integration of independently written code becomes a manual, error-prone process.
- And perhaps most critically — the team never develops the professional habits that the industry will demand of them the moment they leave academia.

As the same industry professional noted:

> “You will be surprised when you go to a job without knowing Git: you might be disqualified immediately from job listings if you do not know how to use it. When would they learn? As far as I can see with your prospectus, there is no course that will teach them how to use Git.”

This is not hyperbole. It is the observable reality of modern software hiring.

5.7 On Prior Experience and Collaborative Mismatch

Collaboration is not defined by proximity. It is not achieved merely by placing individuals in a group and assigning them a shared deliverable. True collaboration requires a common

foundation — shared standards, shared tooling, shared professional discipline. Without these, what results is not collaboration. It is parallel isolation with a single deadline.

The contributor entered the academic environment with approximately two years of independent programming experience prior to enrollment. This includes the successful completion and sale of four software projects, as well as practical experience in automation and systems integration. This background is not cited as a point of superiority. It is cited because it is the direct reason why the proposed workflow standards were not arbitrary — they were earned.

Having operated in professional development environments prior to formal education, the contributor understood firsthand the compounding costs of unstructured collaboration: lost work, unattributable changes, irreconcilable merge conflicts, and catastrophic scope creep. A self-hosted Gitea instance was provisioned. Accounts were created for every team member. An onboarding plan was prepared. An industry professional with full-stack expertise was consulted. A realistic scope was proposed, argued for, and documented.

Every reasonable measure was taken to build the collaborative foundation that the assignment intended.

It was not enough. The team was unable to align on the proposed standards — not out of malice, but out of a genuine mismatch in technical readiness and professional expectation. Continuing under those conditions would not have produced collaboration. It would have produced chaos with a shared filename.

The decision to work independently was therefore the most collaborative decision available. It preserved the integrity of the deliverable. It removed the friction that was preventing progress. And it allowed the contributor to apply the professional standards that the assignment's rubric rewards — without compromise.

Collaboration, at its best, is a force multiplier. But a force multiplier applied to an unstable foundation does not produce more output. It produces faster failure. The solo workflow was not a retreat from collaboration. It was a refusal to call something collaboration that wasn't.

5.8 On the Justification for an Individual Workflow

There is a particular kind of exhaustion that comes not from overwork, but from sustained, unreceived effort. It is the exhaustion of someone who has built the bridge, explained the bridge, demonstrated the bridge, consulted experts about the bridge — and watched, repeatedly, as

the people the bridge was built for chose to wade through the river anyway.

Every reasonable measure was taken to establish a functional collaborative foundation. A self-hosted Gitea instance was provisioned and accounts were created for every team member. An onboarding plan was prepared, designed to bring complete beginners to functional Git literacy in a single session. An industry professional with full-stack expertise was consulted — not to validate a preference, but to stress-test it. His assessment was unambiguous, and it was shared with the team in full. A realistic project scope was proposed, argued for, and documented. Warnings about integration complexity, scope creep, and the professional consequences of avoiding version control were communicated clearly and repeatedly.

None of it was enough.

Not because the arguments were weak. Not because the evidence was insufficient. But because understanding, ultimately, cannot be installed. It has to be earned — usually through the precise experience of failure that the arguments were designed to prevent. As the consulted professional observed, it is genuinely difficult to convince people to adopt structured tooling when they have not yet felt the pain of working without it. The fire has to burn before people believe it is hot.

The fire was visible. The warnings were given. The infrastructure was built.

Some lessons can only be learned by getting burned.

6 Postscript: The Fire

6.1 On Vindication and the Irrelevance of Credit

There is a particular irony in being proven right by the very outcome you warned against. It does not feel triumphant. It feels quiet — the specific quiet of having already left the room before the fire you predicted finally caught. Vindication, when it arrives without an audience, is indistinguishable from irrelevance. And yet the record exists. The timestamps are immutable. The sequence of events is not a matter of interpretation.

Within hours of the contributor's departure from the group, the events described in the preceding chapter resolved themselves in the most instructive manner possible: through direct, unmediated contact with consequence. No further argument was required. No additional evidence was needed. The group encountered, without the contributor's intervention, the

precise difficulties that had been identified, documented, and communicated in advance.

A team member attempted to use Git independently that same evening. His own words, preserved in the group chat, are the only necessary evidence:

> “I tried fuckin scribblingdabledidoo in GitHub last night and it fucked me harder than 1000 guys fucking bonnie blue.”

What had been dismissed as an unnecessary technicality — characterized, in the team’s own words, as a “bitchass skidledaddlediboo type thing” not worth the time of first-year students — became, upon actual encounter, something that demanded reckoning. The encounter was humbling. The humility, to his credit, was genuine.

The team’s pivot toward Git adoption did not emerge from persuasion. It did not emerge from reflection on the arguments that had been made. It emerged from the specific experience of attempting to work without structured version control and discovering, firsthand, why structured version control exists. The arguments had not changed. The evidence had not changed. What changed was that the team finally encountered the evidence directly, without the buffer of someone else’s warnings to dismiss.

This is precisely the dynamic the contributor had anticipated. The fire has to burn before people believe it is hot. The fire burned. The people believed. The repository was created. Its name — `Peak-ass-repository-ng-mga-maiitim-na-komsay` — is, perhaps, the most honest artifact produced by the entire collaboration. It captures, with accidental precision, exactly the register in which this project was conceived and the professional standards under which it was initially intended to be executed.

The contributor will not be part of that project. She will not receive credit for the Gitea infrastructure she provisioned, the onboarding plan she prepared, the industry consultation she arranged, or the professional standards she attempted, at considerable social cost, to introduce. That credit belongs now entirely to the team that remains.

This is fine. Credit was never the point. The point was always the work. The work will be better for the tools being used correctly, even if the person who insisted on those tools is no longer in the room.

Some bridges are built for others to cross.

6.2 On Why Collaboration Was Already Irretrievable

The contributor's stated reason for departure was, in her own words, as follows:

> "No, once we fought, there's already distrust. I'm betting on malicious compliance on this one, even if not intended. It's hard to cooperate after a fight. I'm out. Just wishing y'all good luck."

This assessment was correct in its conclusion, though the full explanation runs deeper than distrust alone. Distrust is a symptom. The underlying conditions that made collaboration irretrievable were structural, and they predated the conflict by the entire duration of the group's existence.

6.2.1 Incompatible Epistemologies

The contributor updates her positions based on evidence, external consultation, and documented professional consensus. The team updated their positions based on direct, personal experience of pain. These are not inherently incompatible ways of learning — but they are incompatible in real-time collaboration under a deadline.

The contributor was, at every stage, two steps ahead of where the team was willing to go. Every suggestion she made was a prediction of a difficulty they had not yet encountered. And a prediction of difficulty, to someone who has not yet felt that difficulty, does not read as expertise. It reads as pessimism, or condescension, or control. The team would only believe the fire was hot when they touched it. The contributor could see the fire from across the room. These are not compatible positions from which to build a shared codebase.

6.2.2 The Power Dynamic That Could Not Be Unnamed

At a critical moment in the exchange, a team member stated the following:

> "Gaslighting yourself na mas magaling ka samen? Totoo naman mas magaling ka samin. Sorry kamo kung overly ambitious kame, kasi we're not like you."

Once this was said aloud, the dynamic was named. And named dynamics cannot be un-named. Every subsequent suggestion the contributor made would have been received not as collaborative input, but as the more skilled person directing the less skilled ones — regardless of her tone, her framing, or her intent. The admission did not clear the air. It poisoned

the well it was attempting to drain. Collaboration under those conditions would have been, at best, a performance of equality over an acknowledged hierarchy. That is not collaboration. That is managed inequality with a shared deadline.

6.2.3 The Exhaustion of the Only Bridge

A third structural failure is worth naming, and it belongs neither to the contributor nor to the most vocal team member. It belongs to `Q` (an alias) — the individual who functioned, throughout the documented exchange, as the group's sole emotional regulator. He pushed for Git adoption. He de-escalated hostility. He advocated for the apology. He held the group together through its most acute phase of conflict.

This is admirable. It is also unsustainable. `Q` was simultaneously managing a resistant team member, absorbing the technical arguments the contributor had made, translating them into terms the group could receive, and attempting to maintain group cohesion — all without any structural support. He was the bridge between two irreconcilable positions, and he was doing it alone, in a group chat, in real time.

A team that depends on a single individual's emotional labor to remain functional is not a team. It is a temporary arrangement held together by one person's willingness to absorb what the structure cannot. That willingness has a limit. The contributor, in departing, did not leave a team. She left `Q`.

6.2.4 On the Selective Tolerance for Criticism

Perhaps the most precise illustration of why continued collaboration was untenable is a contradiction embedded in the documented exchange itself. Early in the conflict, the team communicated the following sentiment to the contributor:

> “The way I see this message is you're limiting what we can do or understand whether we learn it on the easy way or the hard way, we're learning. And we won't regret any choice we made the moment we formed this group.”

The implicit message here is clear: do not criticize our choices. We will learn on our own terms. Your structured concerns are unwelcome.

And yet, within the same exchange, the team's most vocal member reacted to the contributor's calibrated, evidence-based arguments with escalating hostility — accusing her of gaslighting, invoking masculinity as a behavioral standard, and expressing paranoia about external

judgment. The pattern is consistent: criticism delivered through shared suffering is romantic. Criticism delivered through structured argument is an attack.

This is not a personality quirk. It is an epistemological position, and it is one that is fundamentally incompatible with the practice of software engineering, which is, at its core, a discipline of structured criticism — of code, of architecture, of scope, of decisions. A team that cannot receive structured argument cannot function as an engineering team. It can only function as a social group that occasionally writes code.

The contributor was not trying to limit the team. She was trying to practice the discipline they had all ostensibly enrolled to study. That the practice felt like limitation is the most honest possible measure of the distance between them.

6.3 On the Language of Conflict and What It Reveals

A technical disagreement, at its core, is a disagreement about facts. It can be resolved by examining evidence, consulting expertise, and updating one's position in response to new information. This is the ordinary mechanism of intellectual progress, and it is neither complicated nor particularly painful when all parties are operating in good faith.

What occurred in the course of this collaboration was not, ultimately, a technical disagreement. It began as one. It did not remain one for long.

The transition from technical disagreement to personal conflict is always revealing. The specific shape of the hostility — the precise words chosen when composure fails — functions as a diagnostic. It tells the observer not merely that a person is angry, but what assumptions, hierarchies, and values sit beneath the anger, waiting to be exposed.

In this case, the diagnostic was unusually legible.

6.3.1 On the Use of Racial Slurs

Over the course of the exchanges documented in this submission, multiple team members employed racial slurs in casual, unreflective communication. These were not deployed in extremis, as the regrettable output of genuine emotional crisis. They were threaded through ordinary conversation — used to address peers, to punctuate arguments, to fill the rhetorical space where more considered language might otherwise go.

This is worth stating plainly, because it is easy to allow such details to recede into the background of a larger conflict. They should not. The casual use of racial slurs in a collaborative

academic environment is not a stylistic preference. It is not a generational quirk or a marker of in-group familiarity. It is a failure of the most basic professional register — the register that any practitioner in any discipline is expected to maintain when engaging with colleagues, regardless of the perceived informality of the medium.

The irony is not subtle. The same individuals who objected to the contributor's consultation of external practitioners — on the grounds that outsiders lacked the context to offer valid opinions — demonstrated, in the same breath, a comfortable fluency in language that would disqualify them from professional consideration in any environment they might eventually seek to enter. The concern was not that their code would be judged. The concern, apparently, was that their group dynamics would be. The language they used in those group dynamics suggests the concern was not misplaced.

6.3.2 On Being Instructed to Behave "Like a Man"

Perhaps the most structurally revealing moment in the entire exchange was a directive issued to the contributor during the height of the conflict. The instruction, rendered verbatim, was to "fix this problem like a man."

It is worth addressing this directive on two levels: what it assumes about the person it was directed at, and what it reveals about the person who issued it — independent of the first.

On the first level, a brief clarification is warranted. The contributor is a transgender woman, currently in an early and private stage of her transition. Her gender presentation at the time of this exchange did not outwardly signal this. The directive was therefore not issued in knowingly bad faith with respect to her identity. The speaker made an assumption that, from the outside, was not unreasonable to make. He was, nonetheless, wrong. The contributor is a woman. She was a woman throughout every exchange documented in this submission. She will continue to be one. The instruction to behave like a man was issued to someone who has no particular interest in, or obligation to, do so.

This is noted not as an indictment of the speaker's perception — he could not have known — but as a footnote the record deserves, and which the reader may find quietly amusing.

On the second level, however, the speaker's ignorance of the contributor's identity is entirely beside the point. The problem with "fix this problem like a man" is not that it was misdirected. The problem is that it was issued at all.

The invocation of masculinity as a behavioral standard — particularly in the context of conflict

resolution — is a textbook expression of toxic masculinity: the set of cultural norms that equates strength with emotional suppression, competence with dominance, and the resolution of disagreement with the performance of aggression. To instruct someone to handle a conflict "like a man" is to imply that the appropriate response to being challenged is not reflection, not dialogue, not the honest assessment of one's own position — but the forceful reassertion of it. It is to mistake stubbornness for strength and volume for authority.

This framework does not produce good engineers. It does not produce good collaborators. It produces exactly the dynamic that characterized this conflict: a team that received structured, evidence-based feedback and responded with escalating defensiveness, Shakespearean deflection, and paranoia about home addresses — before arriving, belatedly and independently, at the conclusions they had refused to accept when offered them directly.

The irony, then, is complete. The instruction to "fix this problem like a man" was issued by someone who, by the end of the documented exchange, had responded to the problem by writing a multi-paragraph apology in Early Modern English, on his honour and by God, to a woman he did not know was a woman.

None of the assumptions embedded in that directive survived contact with a Git repository.

6.4 On Escalation, Paranoia, and the Self-Revealing Nature of Fear

Among the more structurally interesting moments in the documented exchange is the following, reproduced here in the original:

> "Of course magbibigay yan ng opinions ridiculing us kasi biased yan eh, sana kung kaklase naten mga nagsasabi nyan, eh hinde naman, hell baka bukas buong address namen na leak na dyan."

A careful reading of this statement is instructive. The concern expressed is not that the contributor would be harmed. The concern is that the team's addresses might be leaked — by the external server the contributor had consulted. The fear was not of the contributor's retaliation. It was of the contributor's audience.

This is worth sitting with. The team had, in the preceding exchanges, characterized external opinions as invalid on the grounds that outsiders lacked sufficient context. They were not

classmates. They did not know the full picture. Their perspectives were therefore dismissible. And yet, in the same breath, those same outsiders were imagined as a credible enough threat to warrant concern about personal safety.

The contradiction is not incidental. It reveals the precise shape of the anxiety underneath the argument. The external practitioners were not actually dismissed as irrelevant. They were recognized, implicitly, as an audience capable of forming damaging judgments — judgments the team did not wish to be subjected to. The objection was never really about context. It was about exposure.

The leap from "people on a Discord server are discussing our group project" to "our home addresses may be published tomorrow" is not a proportionate response to the situation. It is the response of someone whose composure has failed and whose threat model has become untethered from the available evidence. It is also, as a matter of record, the response of someone who had spent the preceding exchanges insisting that the situation was entirely manageable and that the team's bonds would carry them through. The confidence and the paranoia existed simultaneously, in the same conversation, within minutes of each other. This is not a contradiction that requires extensive commentary. It speaks adequately for itself.

6.5 On Self-Awareness and Who Gets to Define Competence

The contributor has, at multiple points in this account, the opportunity to claim the high ground entirely. She has declined to do so, because the account would be incomplete without the following acknowledgment.

She has said, directly and without prompting:

> "I came to consult them. I needed external validation whether I even did the right thing. Which admittedly, I did some things wrong. I crowdsourced a verdict instead of just talking to a trusted person. But then again, I still stand by the concerns I raised."

And further:

> "The point is... wishful thinking bad. That's it. Admittedly, I like certainty over the uncertain waters. And I'm in the wrong for being controlling. And there's no fixing optimism over realism. I can't reason over someone optimistic."

This self-assessment is, notably, more rigorous than anything produced by the other parties in the exchange. The contributor identified her own errors — the crowdsourced verdict, the controlling tendency, the preference for certainty that can read as inflexibility — and named them without being asked to. She did not wait for the team to extract the concession. She offered it.

This is the difference between accountability and defensiveness. One of them produces growth. The other produces Shakespearean monologues about pride, delivered after the damage is done.

The contributor's self-criticism does not, however, invalidate her central argument. Being controlling in one's communication style is a real limitation. It does not make version control optional. Being overly certain in one's approach is a real limitation. It does not make a social media platform a realistic four-week deliverable for a team of first-year students without backend experience. The flaw in the delivery does not transfer to the flaw in the message. These are separable, and it is important to separate them.

Her girlfriend's observation is perhaps the cleanest summary available: it is not the contributor's job to be responsible for cleaning up after them. This is true. It is also, implicitly, a recognition that the contributor had been attempting exactly that — and that the attempt, however well-intentioned, was neither sustainable nor within her remit to impose.

6.6 On Leaving as an Act of Integrity

It is necessary to address, directly, the characterization of the contributor's departure as a failure of commitment, a refusal to cooperate, or an assertion of superiority over her peers.

None of these characterizations is accurate.

The contributor left the group after exhausting, in documented sequence, every available avenue for establishing a functional collaborative foundation. She provisioned infrastructure. She prepared onboarding materials. She consulted industry expertise. She proposed a realistic scope. She communicated concerns clearly, repeatedly, and with specific reference to professional standards that are not matters of personal preference but of industry consensus. She offered, explicitly, to teach the required tooling one-on-one, at whatever pace the team required.

Each of these efforts was met with resistance. The resistance was not always hostile — much of it was simply the inertia of people who had not yet encountered the consequences of the choices they were making. But inertia and hostility produce the same outcome in a project context: stasis. And stasis, in a four-week project with an ambitious scope, is not neutral. It is failure in slow motion.

It was also made from a position of self-awareness. The contributor recognized that continuing to operate within a team whose standards, tooling, and professional expectations were irreconcilably misaligned with her own would not produce better work. It would produce conflict, compromise, and a deliverable that satisfied no one. The solo workflow was not a retreat from collaboration. It was a refusal to apply the label of collaboration to something that did not meet its definition.

The departure was the correct decision. The subsequent trajectory of the group — the humbling encounter with Git, the Shakespearean apology, the eventual repository creation — does not retroactively make staying the correct decision. It confirms, instead, that the diagnosis was accurate, the prognosis was accurate, and the only variable that was not accurately predicted was the speed at which the team would arrive, independently, at the conclusions they had refused to accept when offered them.

They got there. It simply required the fire.

6.7 Closing Remarks: An Honest Admission

Let us be direct about what this document actually is.

It is, at its core, an exercise in airing out emotions through structured arguments. The clinical register, the shadequotes, the academic framing — these are the shape that processing takes when the person doing the processing happens to think in structured arguments. The anger was real. The argument is also real. Both things are true simultaneously, and pretending otherwise would be the least honest thing in an otherwise reasonably honest document.

The group existed for less than 48 hours before this was written. That is worth sitting with.

And yet — the arguments hold. The engineering principles are not invalidated by the speed at which they were deployed. Version control is still necessary. Scope discipline is still foundational. The fact that this document was written partly out of frustration does not make it wrong. It makes it human. What structured thinking offers, even when emotionally motivated,

is the ability to separate what is felt from what is true — and to find, in the process of writing, that some of what is felt is also true, and some of it is just noise.

There is also something this environment offered that no controlled, professional setting ever could: the full, unfiltered chaos of a group of people at genuinely different stages of formation, with genuinely different relationships to the discipline, colliding in real time. Curated collaborations — with people who share the same experience level, the same tooling preferences, the same professional expectations — are comfortable. They are also incomplete as a training ground. They do not prepare anyone for the most vocal team member who writes Shakespearean English under pressure, or the emotional regulator of the group who holds everything together through sheer emotional labor, or the first-year student who is not yet sure if he will even continue this course:

> “We’re just first years who don’t even know if we’ll continue this course or not.”

That sentence, dismissed at the time as an excuse, is actually something more honest than most of what was said in the exchange. It is someone naming, plainly, that they do not yet know if this is their discipline. That uncertainty is not a character flaw. It is the ordinary condition of someone at the beginning. The contributor forgot, briefly, what it felt like to be at the beginning — before the projects, before the sales, before the two years of independent work that made Git feel obvious rather than daunting. That forgetting was its own kind of failure.

Closure, it turns out, was never available from the outside. Not from the group, not from the Discord server, not from the people who read this document and agreed, not even from the structured argument itself. The writing did not produce closure. It produced clarity — which is a different and arguably more useful thing. Clarity about what happened. Clarity about what to do differently. Clarity about the distance between being right and being heard, and the importance of not mistaking one for the other.

Next time: bring the tools. Deliver them more gently. Leave earlier if they are refused. And remember that the person who does not yet know if they will continue the course is not an obstacle. They are someone who has not yet been given a reason to stay.

The contributor leaves this project with her codebase intact, her professional standards uncompromised, and her understanding of collaboration — including its failures, and her own contribution to them — meaningfully expanded.

She also leaves as the only person in this account who knew, the entire time, that she was a woman — a fact that, in retrospect, explains rather a lot. :3

The fire was visible. The warnings were given. The infrastructure was built.

Some lessons can only be learned by getting burned.

Some bridges are built for others to cross.

And some people, it turns out, will only pick up the tools after the person who offered them has already left the room.

Part IV

Back Matter

7 Attributions

7.1 Fonts

The fonts used in this project are the following:

Microsoft Corporation. (2007). Microsoft Core Fonts for the Web (Arial) [Font]. Licensed under the Microsoft End-User License Agreement (personal, non-commercial use). Obtained on Fedora Linux via the `msttcore-fonts-installer` package, which downloads the official font files from a Microsoft-authorised mirror in compliance with the EULA. Retrieved May 17, 2026, from https://sourceforge.net/projects/corefonts/.

Sharanda M. (2018). Manrope [Font]. Licensed under the SIL Open Font License, Version 1.1. Retrieved May 16, 2026, from https://fonts.google.com/specimen/Manrope.

Voos M. (2020). Manrope Italic [Font]. Licensed under the SIL Open Font License, Version 1.1. Retrieved June 14, 2025, from https://github.com/malte-v/manrope-italic.

Miwa, S. (2018). Comic Mono [Font]. Licensed under the MIT License. Retrieved June 5, 2025, from https://github.com/dtinth/comic-mono-font. Modified by dtinth (2019).

7.2 AI Disclosure

In the interest of academic integrity, the following portions of this project were generated or heavily modified with the assistance of artificial intelligence tools:

- **HTML Templates:** All HTML files under the `src/templates/bookmarks` directory were generated by OpenAI Codex.
- **Frontend Scripts:** The majority of the JavaScript code within the `src/static/scripts` directory was generated by Anthropic's Claude AI.
- **Recent Backend & Frontend Modifications:** The `edit` bookmark functionality, updates to `src/api/bookmarks.py`, `src/static/scripts/bookmarks.js`, the authentication tightening in `main.py`, the `/login` auto-redirect endpoint, the hamburger menu styling, the SVG CSS `mask-image` icon theming technique in `base.css` and `base.j2.html`, comprehensive CSS accessibility fixes to replace hardcoded colors with theme variables across `style.css`, `bookmarks.css`, `code.css`, and `base.css`, and the removal of deprecated login templates and CSS redundancies were recently generated and modified by an AI

coding assistant (Google DeepMind's Antigravity).

Bibliography

Abrams, D., Baecker, R., & Chignell, M. (1998). Information archiving with bookmarks: Personal web space construction and organisation [One of the foundational studies on how users construct and manage personal bookmark collections.]. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 41–48. https://doi.org/10.1145/274644.274651

Cegłowski, M. (July 2009). Pinboard: Social bookmarking for introverts [Founding announcement and philosophy statement for Pinboard.in.]. https://blog.pinboard.in/

Cegłowski, M. (July 2015). Pinboard turns six [Announcement of the switch from a one-time incrementing signup fee to a flat $11/year subscription model.]. https://blog.pinboard.in/2015/07/pinboard_turns_six/

Chui, M., Manyika, J., Bughin, J., Dobbs, R., Roxburgh, C., Sarrazin, H., Sands, G., & Westergren, M. (July 2012). The social economy: Unlocking value and productivity through social technologies (tech. rep.) (Reports that knowledge workers spend approximately 20% of their working week searching for internal information or tracking down colleagues who can help with specific tasks.). McKinsey Global Institute. https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-social-economy

Jones, W. (2007). Keeping found things found: The study and practice of personal information management [A comprehensive treatment of how people manage personal information; directly relevant to the organizational challenge that bookmark managers attempt to solve.]. Morgan Kaufmann Publishers.

Noble, J. (2021). Johnny.decimal: A system to organise your life [Primary reference for the Johnny.Decimal organizational system. The system assigns unique two-part numeric IDs (e.g., 15.23) to every item in a collection, using areas and categories to impose navigable structure.]. https://johnnydecimal.com

Pernice, K. (2014). Navigation: You are here [Foundational UX research on navigation patterns; the quote cited is a paraphrase of Nielsen's widely cited principle regarding cross-site consistency.]. https://www.nngroup.com/reports/navigation-you-are-here/

Raindrop.io. (2024). Raindrop.io: All-in-one bookmark manager [Official product documentation and feature listing for Raindrop.io, including pricing tiers and platform availability.]. https://raindrop.io

Singel, R. (December 2010). Yahoo axes delicious, the web's favorite bookmarking service [Reports on Yahoo's decision to sunset Delicious, which drove mass migration to Pinboard.]. https://www.wired.com/2010/12/yahoo-delicious/

Sissbruecker. (2020). Linkding: Self-hosted bookmark manager [Primary source for linkding's feature set, architecture, and self-hosting requirements. The project is designed to be minimal, fast, and set up using Docker.]. https://github.com/sissbruecker/linkding

Tsai, M. (September 2024). The end of pinboard? [Community discussion thread documenting Pinboard's operational decline in 2024, including API throttling, extended outages, and developer unresponsiveness.]. https://mjtsai.com/blog/2024/09/20/the-end-of-pinboard/